

Atividade Prática — API de Biblioteca com Node.js e Sequelize

Desenvolvimento Web 2

Modalidade

Atividade em dupla

Tema

Desenvolvimento de uma API REST utilizando **Node.js, Sequelize e SQLite**.

1. Objetivo da atividade

Nesta atividade, vocês deverão desenvolver uma pequena API para gerenciamento de uma **biblioteca**.

O objetivo é aplicar, na prática, os principais conceitos estudados:

- Configuração do Sequelize;
 - Conexão com banco de dados SQLite;
 - Criação de Models;
 - Tipos de dados;
 - Validações;
 - CRUD;
 - Consultas com filtros;
 - Relacionamentos entre tabelas;
 - Uso de `include`;
 - Organização do código.
-

2. Cenário

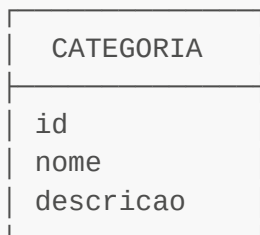
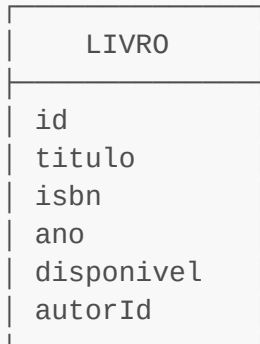
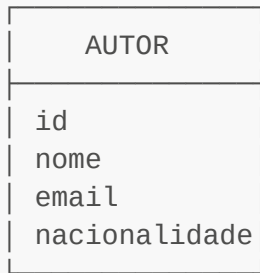
Uma biblioteca deseja informatizar o cadastro de seus **autores, livros e categorias**.

O sistema deverá permitir cadastrar autores e livros, organizar os livros em categorias e realizar consultas utilizando os relacionamentos entre as entidades.

A dupla deverá desenvolver o backend responsável por essas operações.

3. Entidades do sistema

O sistema deverá possuir inicialmente as seguintes entidades:



4. Relacionamentos

Autor e Livro

Um autor pode escrever vários livros.

Um livro pertence a apenas um autor.



▼
autorId

No Sequelize, vocês deverão utilizar:

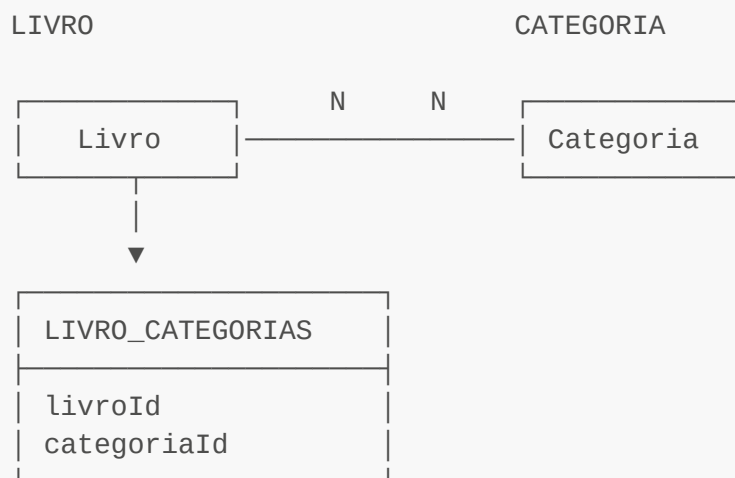
```
Autor.hasMany(Livro, {
  foreignKey: "autorId"
});

Livro.belongsTo(Autor, {
  foreignKey: "autorId"
});
```

Livro e Categoria

Um livro pode pertencer a várias categorias.

Uma categoria pode possuir vários livros.



Utilizem:

```
Livro.belongsToMany(Categoria, {
  through: "livro_categorias"
});

Categoria.belongsToMany(Livro, {
  through: "livro_categorias"
});
```

5. Estrutura esperada do projeto

Organizem o projeto utilizando a seguinte estrutura:

```
biblioteca-api/  
├── database/  
│   └── biblioteca.sqlite  
├── src/  
│   ├── config/  
│   │   └── database.js  
│   ├── models/  
│   │   ├── Autor.js  
│   │   ├── Livro.js  
│   │   ├── Categoria.js  
│   │   └── index.js  
│   ├── repositories/  
│   │   ├── AutorRepository.js  
│   │   ├── LivroRepository.js  
│   │   └── CategoriaRepository.js  
│   ├── services/  
│   │   ├── AutorService.js  
│   │   ├── LivroService.js  
│   │   └── CategoriaService.js  
│   ├── controllers/  
│   │   ├── AutorController.js  
│   │   ├── LivroController.js  
│   │   └── CategoriaController.js  
│   ├── routes/  
│   │   ├── autorRoutes.js  
│   │   ├── livroRoutes.js  
│   │   └── categoriaRoutes.js  
│   └── app.js  
├── package.json  
└── README.md
```

A dupla poderá realizar adaptações na estrutura, desde que mantenha uma separação clara de responsabilidades.

6. Etapa 1 — Configuração do projeto

Inicializem o projeto:

```
npm init -y
```

Instalem as dependências necessárias:

```
npm install express sequelize sqlite3
```

Criem a conexão com o banco SQLite.

Arquivo:

```
src/config/database.js
```

Exemplo de configuração:

```
const { Sequelize } = require("sequelize");

const sequelize = new Sequelize({
  dialect: "sqlite",
  storage: "./database/biblioteca.sqlite",
  logging: false
});

module.exports = sequelize;
```

Critério

A aplicação deverá conseguir estabelecer conexão com o banco.

7. Etapa 2 — Criando os Models

Implementem os seguintes Models:

Autor

Campos obrigatórios:

Campo	Tipo	Regra
<code>id</code>	INTEGER	Chave primária
<code>nome</code>	STRING	Obrigatório
<code>email</code>	STRING	Obrigatório e único
<code>nacionalidade</code>	STRING	Opcional

Requisito adicional

O campo `email` deve possuir validação de formato.

Livro

Campos:

Campo	Tipo	Regra
<code>id</code>	INTEGER	Chave primária
<code>titulo</code>	STRING	Obrigatório
<code>isbn</code>	STRING	Obrigatório e único
<code>ano</code>	INTEGER	Obrigatório
<code>disponivel</code>	BOOLEAN	Valor padrão <code>true</code>
<code>autorId</code>	INTEGER	Obrigatório

Categoria

Campos:

Campo	Tipo	Regra
<code>id</code>	INTEGER	Chave primária
<code>nome</code>	STRING	Obrigatório e único
<code>descricao</code>	TEXT	Opcional

8. Etapa 3 — Relacionamentos

Implementem corretamente os relacionamentos:

Autor 1 ————— N Livro

e:

```
Livro N ————— N Categoria
```

A aplicação deverá criar corretamente as chaves e a tabela associativa.

9. Etapa 4 — CRUD de Autores

Implementem as seguintes operações.

Criar autor

```
POST /autores
```

Exemplo:

```
{
  "nome": "Machado de Assis",
  "email": "machado@email.com",
  "nacionalidade": "Brasileiro"
}
```

Listar autores

```
GET /autores
```

Buscar autor por ID

```
GET /autores/:id
```

Caso não exista:

```
{
  "erro": "Autor não encontrado"
}
```

Atualizar autor

```
PUT /autores/:id
```

Excluir autor

```
DELETE /autores/:id
```

10. Etapa 5 — CRUD de Livros

Implementem:

```
POST    /livros
GET     /livros
GET     /livros/:id
PUT     /livros/:id
DELETE  /livros/:id
```

Exemplo de cadastro:

```
{
  "titulo": "Dom Casmurro",
  "isbn": "9780000000001",
  "ano": 1899,
  "autorId": 1
}
```

11. Etapa 6 — Consulta com relacionamento

A rota:

```
GET /livros
```

deve retornar os livros juntamente com seus autores.

Utilizem o recurso:

```
include
```

Exemplo de resultado esperado:

```
[
  {
    "id": 1,
    "titulo": "Dom Casmurro",
    "ano": 1899,
    "disponivel": true,
    "Autor": {
      "id": 1,
      "nome": "Machado de Assis"
    }
  }
]
```

12. Etapa 7 — Categorias

Implementem o CRUD básico:

```
POST    /categorias
GET     /categorias
GET     /categorias/:id
PUT     /categorias/:id
DELETE  /categorias/:id
```

13. Etapa 8 — Associando livros e categorias

Criem uma rota para associar uma categoria a um livro.

Sugestão:

```
POST /livros/:livroId/categorias/:categoriaId
```

Exemplo:

```
POST /livros/1/categorias/2
```

O resultado esperado é a criação do relacionamento:

```
Livro: 1  
  |  
  ▼  
Categoria: 2
```

14. Etapa 9 — Busca avançada

Implementem filtros na rota:

```
GET /livros
```

A rota deverá aceitar opcionalmente:

```
GET /livros?titulo=dom
```

```
GET /livros?ano=1899
```

```
GET /livros?disponivel=true
```

Também deverá ser possível combinar filtros:

```
GET /livros?titulo=dom&disponivel=true
```

Utilizem o Sequelize para construir as condições.

15. Etapa 10 — Paginação

Adicionem suporte para:

```
GET /livros?page=1&limit=10
```

A resposta deve conter informações da paginação.

Exemplo:

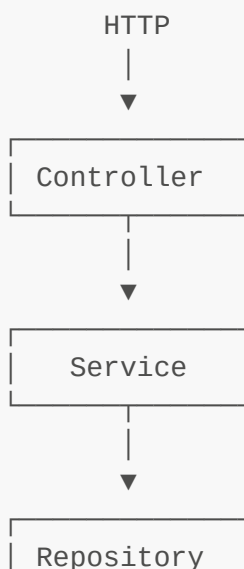
```
{
  "data": [
    {
      "id": 1,
      "titulo": "Dom Casmurro"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 50,
    "totalPages": 5
  }
}
```

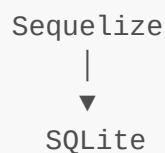
A dupla deverá calcular corretamente:

```
const offset = (page - 1) * limit;
```

16. Etapa 11 — Organização das responsabilidades

A aplicação deverá seguir, preferencialmente, o seguinte fluxo:





Controller

Responsável por:

- Receber requisições;
- Ler parâmetros;
- Chamar o Service;
- Retornar respostas HTTP.

Service

Responsável por:

- Regras de negócio;
- Validações específicas da aplicação;
- Coordenação das operações.

Repository

Responsável por:

- Consultas ao banco;
- Operações do Sequelize;
- Acesso aos Models.

17. Desafio extra — Empréstimo de livros

Para as duplas que concluírem a atividade principal, implementem:

USUARIO

e:

EMPRESTIMO

Um empréstimo deverá possuir:

```
id
usuarioId
livroId
dataEmprestimo
dataDevolucao
status
```

Regras do empréstimo

Um livro só poderá ser emprestado se:

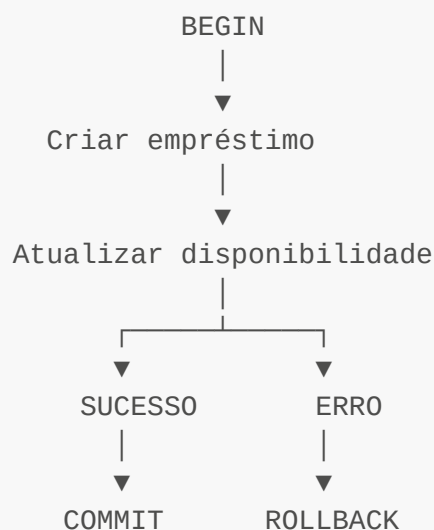
```
disponivel = true
```

Ao criar o empréstimo:

```
① Criar empréstimo
  |
  ▼
② Alterar livro
  |
  ▼
disponivel = false
```

Essas operações devem ocorrer utilizando uma:

TRANSAÇÃO



Importante

Ao final, os dois integrantes devem:

- Revisar todo o código;
- Integrar as funcionalidades;
- Executar e testar a aplicação juntos;
- Ser capazes de explicar qualquer parte da solução.

A divisão não significa que cada integrante será avaliado apenas pela sua parte.

18. Entregáveis

A dupla deverá entregar no classroom o link do repositório github:

1. Código-fonte

Projeto completo contendo:

- ✓ Node.js
- ✓ Express
- ✓ Sequelize
- ✓ SQLite
- ✓ Models
- ✓ Relacionamentos
- ✓ CRUD
- ✓ Consultas
- ✓ Paginação

2. README.md

O projeto deve possuir um README contendo:

Nome dos integrantes

Integrante 1:
Integrante 2:

Como instalar

```
npm install
```

Como executar

```
npm start
```

Rotas disponíveis

Exemplo:

Método	Rota	Descrição
POST	/autores	Criar autor
GET	/autores	Listar autores
GET	/autores/:id	Buscar autor
PUT	/autores/:id	Atualizar autor
DELETE	/autores/:id	Excluir autor
POST	/livros	Criar livro
GET	/livros	Listar livros
GET	/livros/:id	Buscar livro
PUT	/livros/:id	Atualizar livro
DELETE	/livros/:id	Excluir livro

19. Critérios de avaliação

Critério	Pontos
Configuração correta do Sequelize	1,0
Models e tipos de dados	1,0
Validações e constraints	1,0
CRUD funcionando corretamente	2,0
Relacionamentos	1,5
Consultas com <code>include</code>	1,0
Filtros e paginação	1,0
Organização e qualidade do código	1,0
README e documentação	0,5
TOTAL	10,0

20. Checklist da dupla

Antes da entrega, verifiquem:

- ☐ Projeto executa sem erros
- ☐ Banco SQLite foi criado
- ☐ Sequelize conecta corretamente
- ☐ Models foram implementados
- ☐ Campos obrigatórios possuem validação
- ☐ E-mails possuem formato válido
- ☐ Campos únicos possuem constraint
- ☐ CRUD de Autor funciona
- ☐ CRUD de Livro funciona
- ☐ CRUD de Categoria funciona
- ☐ Relacionamento Autor → Livro funciona
- ☐ Relacionamento Livro ↔ Categoria funciona
- ☐ include retorna dados relacionados
- ☐ Filtros funcionam
- ☐ Paginação funciona
- ☐ Erros são tratados
- ☐ Código está organizado
- ☐ README foi criado
- ☐ Os dois integrantes conhecem a solução

Pergunta final

Ao terminar a atividade, a dupla deve conseguir explicar:

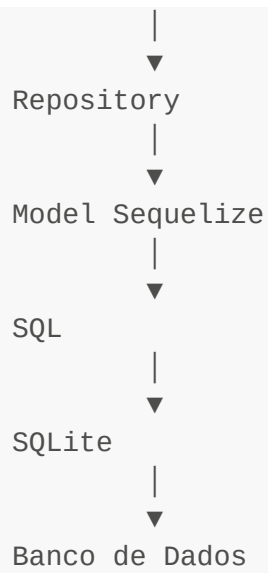
Como um objeto JavaScript...

```
{  
  titulo: "Dom Casmurro"  
}
```

...se transforma em um registro no banco?

O caminho esperado é:

```
Objeto JavaScript  
  |  
  ▼  
Controller  
  |  
  ▼  
Service
```



Bom trabalho!

Objetivo final: não apenas fazer funcionar, mas entender como e por que funciona.